Exploratory Data Analysis: Northwind Lineage Benchmark

1. Dataset

We use the **Northwind lineage benchmark** released with the core paper (Dutkiewicz et al., github.com/dudenzz/lineage). It extends Microsoft's Northwind sample database with derived objects (views, staging, temporary and final tables) created by SQL lineage scenarios, and records every derivation in a **row-level lineage log**, which is our ground truth. Valentine compares tables and columns, so we aggregate the log to **table level**: A → B is a link when at least one row of B was derived from A. The benchmark has three splits:

split	objects	Northwind / derived / unlogged	log rows	table-level links	joins (targets with >1 source)	evaluable links (direct + bridged)
train	27	13 / 10 / 4	520	13	2	9 + 2
test	17	13 / 4 / 0	269	6	1	4 + 1
multi	16	4 / 12 / 0	9,274	14	1	14 + 0

2. Basic analysis

Tables range from 0 to 2,155 rows (OrderDetails); derived objects have 1–830 rows and 2–6 columns. Most columns are text (train: 72 text, 44 numeric, 8 date). Missing values are rare except in Employees (20%) and Suppliers (17%). Lineage chains have up to four links and every split contains joins (Fig. 1). We found five data-quality issues, all handled by our loader (src/lineage_data.py):

- **Temporary objects without data.** ##Temp… tables appear in the train and test logs but have no file. Links through them are *bridged* (A → ##Temp → B becomes A → B), which gives 3 of the 30 evaluable links.
- **Empty tables.** 6 tables have no rows (4 in train, 2 in test); they cannot be matched on values.
- **Unlogged derived objects.** train contains 4 derived objects whose lineage is not logged (e.g. vwProductSupplierMatch, clearly built from Products). A correct match to them counts as a false positive.
- **Encoding and labels.** 46 values were stored as Python byte strings (b'Toms Spezialit\xe4ten') and are decoded; in multi, the column TableFulfillmentLeads.CustomerID actually holds order IDs.
- **Format differences.** \r vs \n line endings, an extra type row, None vs NULL, and name variants (vw_OrderFulfillment vs vwOrderFulfillment.csv, [Order Details] vs OrderDetails.csv) are normalised.

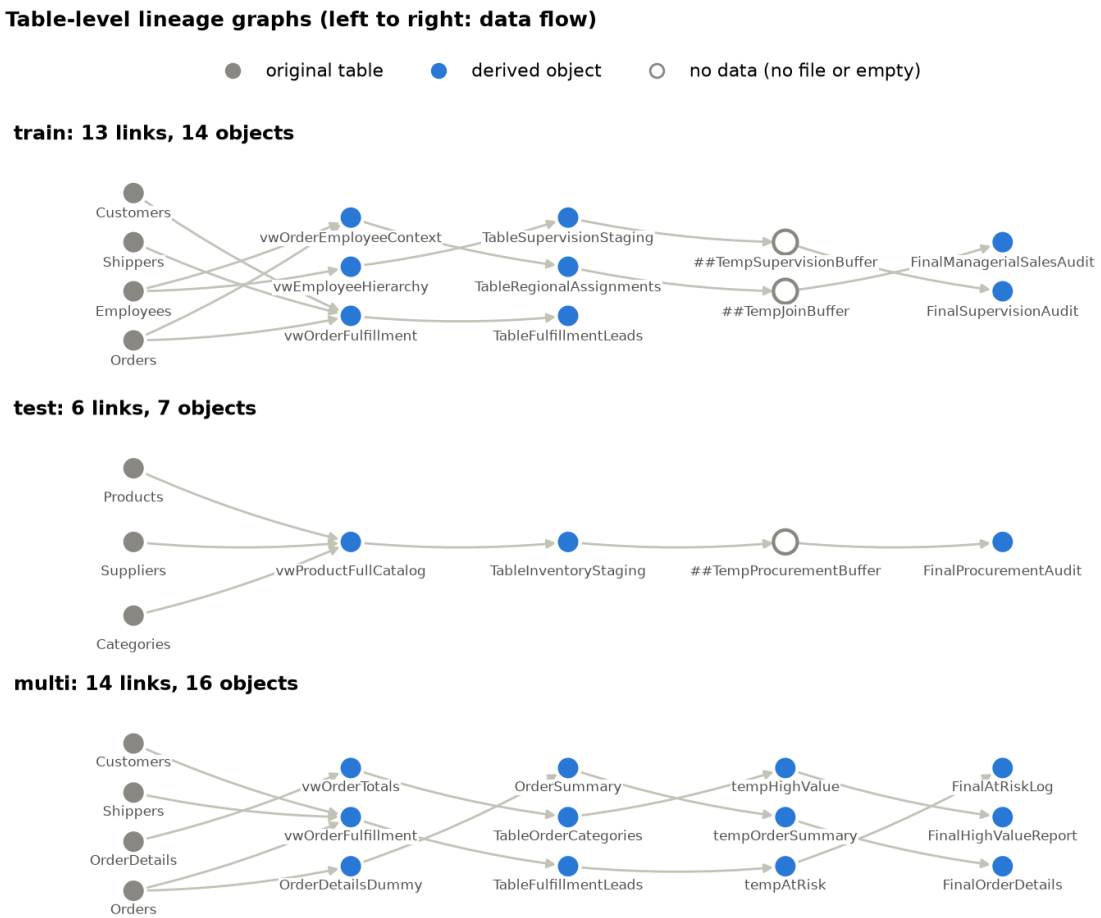


Figure 1. Table-level lineage per split. Hollow nodes have no data, so their links are bridged for evaluation.

3. Specific analysis: is Valentine's evidence in the data?

For all 842 ordered pairs of usable tables (A, B) we approximate the two signals Valentine relies on (src/eda.py). The **value score** is, for each column of B, the share of its distinct values found in some column of A, averaged over B's columns: the signal of instance-based matchers (Jaccard, Distribution-based, COMA with instances). The **name score** is the mean best column-name similarity: the signal of schema-based matchers (COMA, Cupid, Similarity Flooding). Each pair is labelled by its relation in the ground truth.

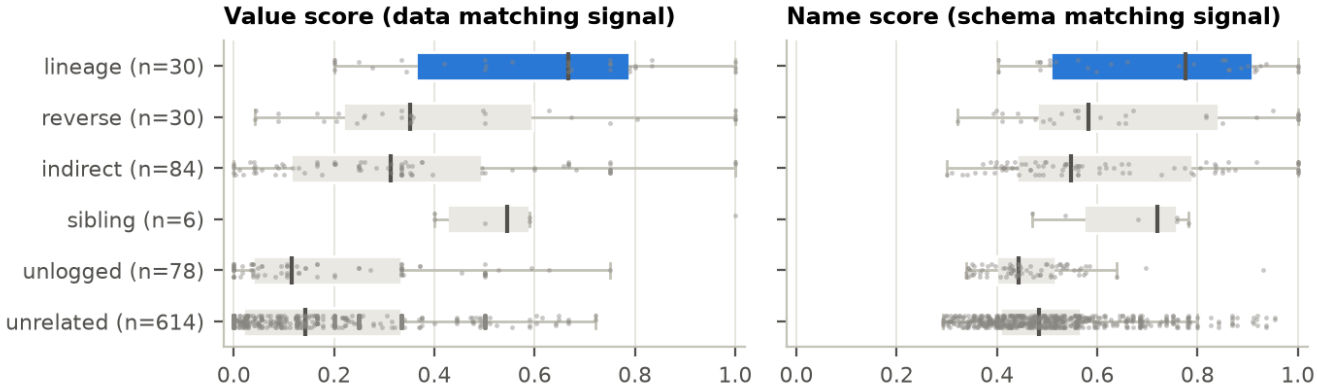


Figure 2. Value and name scores by relation (842 pairs). *reverse*: B → A is the link; *indirect*: a further ancestor; *sibling*: derived from a common source; *unlogged*: involves a derived object without logged lineage.

Shared values separate lineage pairs (median value score **0.67**) from unrelated pairs (**0.14**) much more clearly than column names do (**0.78** vs **0.48**). Reversed links (0.35), indirect ancestors (0.31) and siblings (0.55) lie in between: they are the hard negatives for a table-level prediction.

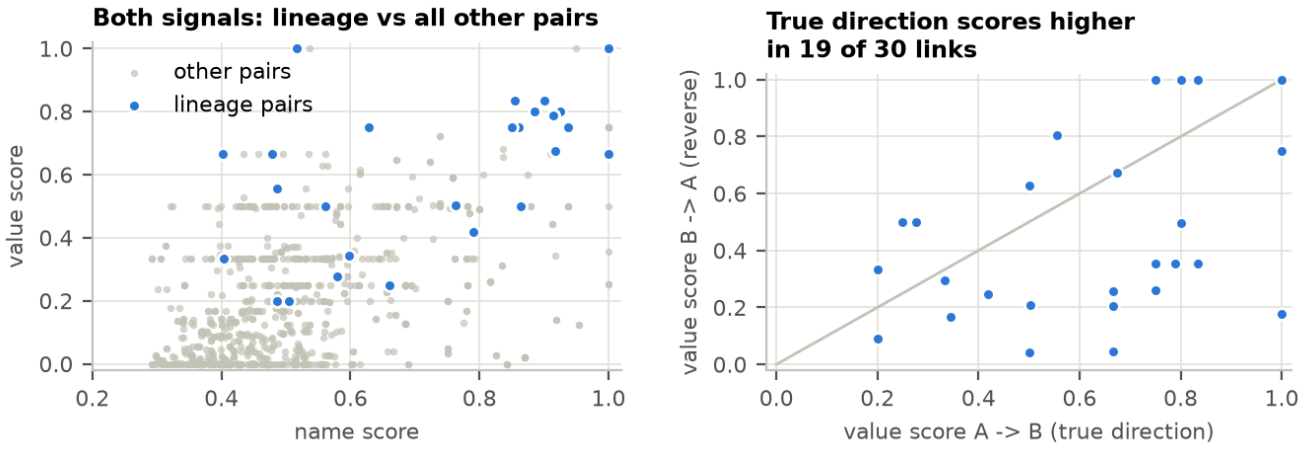


Figure 3. Left: lineage pairs are high on both signals, but many other pairs overlap them. Right: for each lineage link, the value score in the true and in the reverse direction.

What happens to the columns of a derived table? Of the 122 columns of derived tables, **39%** keep both name and values (easy for both matcher types), **20%** are renamed but keep their values (e.g. CompanyName → CustomerName; only data matching can find them) and **41%** are computed or transformed (neither signal helps). **Direction** is hard: the true direction has the higher value score in only 19 of 30 links. **Likely false positives** come from foreign keys (Territories → EmployeeTerritories, 0.72) and small integer keys: in 50% of unrelated pairs at least one column is fully contained in the other table.